Q4. LSTM – Calculate Gates and Hidden State

Question: Assume an LSTM cell with one unit and the following weights and biases:

- Forget gate hidden-state weights: $W_f=[0.1,0.2,0.3]$
- Forget gate input weights: $U_f=[0.3,0.4,0.5]$
- Input gate hidden-state weights: $W_i=[0.5,0.6,0.7]$
- Input gate input weights: $U_i=[0.6,0.7,0.8]$
- Output gate hidden-state weights: $W_o=[0.7,0.8,0.9]$
- Output gate input weights: $U_o=[0.8,0.9,0.7]$
- Cell state hidden-state weights: $W_c=[0.4,0.6,0.5]$
- Cell state input weights: $U_c=[0.3,0.4,0.5]$
- Biases: $b_f=b_i=b_o=b_c=0.1$

Compute forget gate, input gate, output gate, candidate cell state, cell state and hidden state.

Solution:

LSTM mathematical formulas:

1. Forget Gate

$$
f_t=\sigma(W_fh_{t-1}+U_fx_t+b_f)
$$

2. Input Gate

$$
i_t=\sigma(W_ih_{t-1}+U_ix_t+b_i)
$$

3. Output Gate

$$
o_t=\sigma(W_oh_{t-1}+U_ox_t+b_o)
$$

4. Candidate Cell State

$$\tilde{C}_t=\tanh(W_{ch}h_{t-1}+U_{cx}x_t+b_c)$$

5. Cell State

$$C_t=f_tC_{t-1}+i_t\tilde{C}_t$$

6. Hidden State

$$h_t=o_t\tanh(C_t)$$

Where:

- σ = Sigmoid activation function.
- $\tanh$ = Hyperbolic tangent function.

Important: The question does not provide the current input x_t, previous hidden state h_{t-1}, or previous cell state C_{t-1}. Therefore, numerical values cannot be calculated. The above formulas are the complete mathematical solution possible with the given data.

Q7. Calculate Attention Score and Context Representation

Question: Given the following Query, Key and Value vectors:

- Query vector (Q): [3, 5]
- Key vector (K): [2, 4]
- Value vector (V): [6, 12]
- Final attention weight = 0.4

(a) Calculate the raw alignment score using dot product.

Solution:

Formula:

$$
Score=Q\cdot K
$$

Substitute values:

$$
=(3\times2)+(5\times4)
$$

$$
=6+20
$$

$$
\boxed{Score=26}
$$

(b) Calculate final context representation.

Formula:

$$
Context=Attention\ Weight\times Value
$$

$$
=0.4\times[6,12]
$$

$$
=[0.4\times6,\ 0.4\times12]
$$

$$
\boxed{Context=[24,4.8]}
$$

Final Answer

- Raw alignment score = 26
- Context representation = [24, 4.8]

Q1. Calculate Maximum, Minimum and Average Pooling

Question: Given two vectors:

$$
A=[-0.1,-0.2,-0.5,-0.3]
$$

$$
B=[-0.2,-0.4,-0.3,-0.5]
$$

Calculate maximum, minimum and average pooling

Solution:

We perform element-wise pooling on the two vectors.

1. Maximum Pooling

Select the maximum value from each corresponding position.

$$
Max=[\max(-0.1,-0.2),
$$

$$
\[
$$

$$
\max(-0.2,-0.4),
$$

$$
\max(-0.5,-0.3),
$$

$$
\max(-0.3,-0.5)]
$$

$$
\boxed{Max=[-0.1,-0.2,-0.3,-0.3]}
$$

2. Minimum Pooling

Select the minimum value from each corresponding position.

$$
Min=[-0.2,-0.4,-0.5,-0.5]
$$

$$
\boxed{Min=[-0.2,-0.4,-0.5,-0.5]}
$$

3. Average Pooling

Calculate the average of corresponding values.

$$
Avg=\left[
\frac{-0.1-0.2}{2},
$$

\frac{-0.2-0.4}{2},
\frac{-0.5-0.3}{2},
\frac{-0.3-0.5}{2}
\right]
\]

\[
\boxed{Avg=[-0.15,-0.3,-0.4,-0.4]}
\]

Final Answer
Pooling type Result
Maximum [−0.1, −0.2, −0.3, −0.3]
Minimum [−0.2, −0.4, −0.5, −0.5]
Average [−0.15, −0.3, −0.4, −0.4]

Q9 (1). Write Mathematical Formulation for Attention
Question: Write the mathematical formulation for Attention.

Answer:

Attention is a mechanism in deep learning that helps a model focus on the most important information in an input sequence.

Step 1: Calculate similarity score
\[
Score(Q,K)=QK^T
\]

Step 2: Apply SoftMax
\[

\alpha_i=\frac{e^{score_i}}{\sum_j e^{score_j}}
\]

Step 3: Calculate weighted sum of values
\[
Attention(Q,K,V)=\sum_i\alpha_iV_i
\]

Final Formula
\[
\boxed{Attention(Q,K,V)=SoftMax(QK^T)V}
\]

Where:
- Q = Query
- K = Key
- V = Value
- \(\alpha\) = Attention weight

Conclusion: Attention assigns weights to different input elements and focuses on the most relevant information.

Q10 (2). Write Mathematical Formulation for RNN

Question: Write the mathematical formulation for Recurrent Neural Network (RNN).

Answer:

RNN is a type of neural network used to process sequential data such as text, speech and time-series data.

It uses the previous hidden state to remember information from earlier inputs.

1. Hidden State Calculation

$$\boxed{h_t=\tanh(W_{xh}x_t+W_{hh}h_{t-1}+b_h)}$$

Where:

- x_t = Current input
- h_t = Current hidden state
- h_{t-1} = Previous hidden state
- W_{xh} = Input-to-hidden weights
- W_{hh} = Hidden-to-hidden weights
- b_h = Bias

2. Output Calculation

$$\boxed{y_t=\text{SoftMax}(W_{hy}h_t+b_y)}$$

Where:

- y_t = Output
- W_{hy} = Hidden-to-output weights
- b_y = Output bias

3. RNN Architecture

Input Layer (x_t)

Hidden Layer (h_t)

Output Layer (y_t)

The previous hidden state is passed to the next time step.

Conclusion: RNN processes sequential information by maintaining a hidden state that carries information from previous time steps.